

学号： 1906410229

沈阳建筑大学

上机报告

课程名称： 算法设计与分析

项目名称： 分治与动态规划

2022年秋季学期

上机时间：第 6 周 星期 五 第 3 ~ 4 节

学生姓名： 蔡艺洵 专业班级： 计算机 1902

指导教师： 刘俊岭 成 绩：

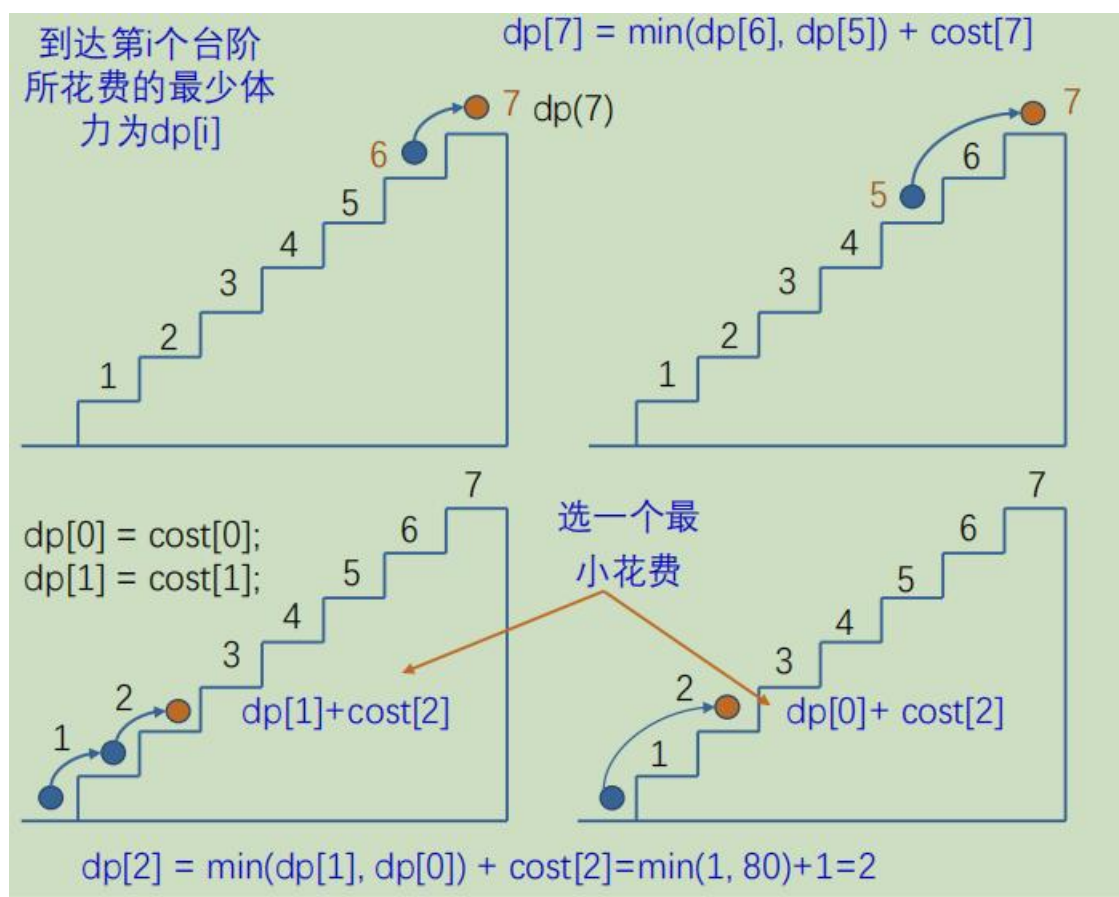
项 目 名 称	分治与动态规划	学 时	4
日期： 2022 年 10 月 7 日		第 6 教学周	
硬件及软件			
名 称		版本、型号	
Dev-C++		5.11	
上机目的与要求： 1. 问题描述，题目定义。 2. 算法实例，实例演示程序过程。 3. 程序代码，完整可运行程序，并加注释。 4. 程序验证，要求有不同的输入、输出，并截图。 5. 算法分析，时间复杂度与空间复杂度。			
本次任务： 实验 3-1：使用最小花费爬楼梯 实验 3-2：走格子求最小（大）路径和 实验 2-5：线性时间选择			

实验 3-1 使用最小花费爬楼梯

1.问题描述:

数组的每个下标作为一个阶梯，第 i 个阶梯对应着一个非负数的体力花费值 $\text{cost}[i]$ （下标从 0 开始）。每当你爬上一个阶梯你都要花费对应的体力值，一旦支付了相应的体力值，你就可以选择向上爬一个阶梯或者爬两个阶梯。请你找出达到楼层顶部的最低花费。在开始时，你可以选择从下标为 0 或 1 的元素作为初始阶梯。要求：1. 给出最小花费；2. 给出所走的台阶编号

2.算法实例:



3.程序代码:

```
/**
 * 使用最小花费爬楼梯，增加了轨迹输出。
 */
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

class Solution {
public:
    int min(int a,int b){
        if(a<b)
            return a;
        else
            return b;
    }
    int minCostClimbingStairs(vector<int>& cost, vector<int>& rec) {
        vector<int> dp(cost.size());
        dp[0] = cost[0];
        dp[1] = cost[1];
        for (int i = 2; i < cost.size(); i++) {
            //dp[i] = min(dp[i - 1], dp[i - 2]) + cost[i];
            if (dp[i - 1] < dp[i - 2]){
                dp[i] = dp[i - 1] + cost[i];
                rec[i] = i - 1;
            }
            else{
                dp[i] = dp[i - 2] + cost[i];
                rec[i] = i - 2;
            }
        }
        // 注意最后一步可以理解为不用花费，所以取倒数第一步，第二步的最少值
        return min(dp[cost.size() - 1], dp[cost.size() - 2]);
    }
};

int main()
{
    vector<int> cost;
    cost.push_back(1);cost.push_back(80);
    cost.push_back(1);cost.push_back(2);
    cost.push_back(1);cost.push_back(90);
```

```

cost.push_back(1);cost.push_back(1);
cost.push_back(100);cost.push_back(1);
// cost.push_back(1);cost.push_back(2);
// cost.push_back(1);cost.push_back(3);
// cost.push_back(6);cost.push_back(9);
// cost.push_back(1);cost.push_back(1);
// cost.push_back(10);cost.push_back(10);
// cost.push_back(2);cost.push_back(1);
vector<int> rec;
for(int k=0;k<10;k++){    //第二个例子此处的 k<12
    rec.push_back(0);
} Solution solution;

cout << solution.minCostClimbingStairs(cost,rec) << endl;
for (int i = 0; i < rec.size(); i++){
    cout << rec[i] << " ";
}
cout << endl;
cout << "轨迹是: ";
int j = rec.size()-1;
cout << j << " ";
while (j!= 0){
    cout << rec[j] << " ";
    j = rec[j];
}
printf("\n");
system("PAUSE");
return 0;
}

```

4.程序验证:

```

vector<int> cost;
cost.push_back(1);cost.push_back(80);
cost.push_back(1);cost.push_back(2);
cost.push_back(1);cost.push_back(90);
cost.push_back(1);cost.push_back(1);
cost.push_back(100);cost.push_back(1);

```

验证爬楼梯体力值花费数组为: cost = [1, 80, 1, 2, 1, 90, 1, 1, 100, 1], 输入为 1, 80, 1, 2, 1, 90, 1, 1, 100, 1, 输出结果为轨迹是: 9 7 6 4 2 0, 结果如图所示:

选择 F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第二次上机课\3-1.exe

```

6
0 0 0 2 2 4 4 6 6 7
轨迹是: 9 7 6 4 2 0
请按任意键继续. . .

```

验证爬楼梯体力值花费数组为: $\text{cost} = [1, 2, 1, 3, 6, 9, 1, 1, 10, 10, 2, 1]$, 输入为 1, 2, 1, 3, 6, 9, 1, 1, 10, 10, 2, 1, 输出结果为轨迹是: 11 9 7 6 4 2 0, 结果如图所示: (该例子是我的个人测试的)

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第二次上机课\3-1.exe

```
21
0 0 0 1 2 3 4 6 6 7 8 9
轨迹是: 11 9 7 6 4 2 0
请按任意键继续. . .
```

5. 算法分析:

该算法使用了动态规划, 是动态规划类算法。首先确定 dp 数组以及下标的含义, 本题中 $\text{dp}[i]$ 的定义: 到达第 i 个台阶所花费的最少体力为 $\text{dp}[i]$ 。然后确定递推公式 状态转移方程 $\text{dp}[i] = \min(\text{dp}[i - 1], \text{dp}[i - 2]) + \text{cost}[i]$; 不要忘记 dp 数组要初始化 $\text{dp}[0] = \text{cost}[0]; \text{dp}[1] = \text{cost}[1]$;

- 时间复杂度: $O(n)$
- 空间复杂度: $O(n)$

实验 3-2 走格子求最小（大）路径和

1. 问题描述:

有一个由数字组成的规格为 $n \times m$ 的矩阵, 初始在左上角, 要求每次只能向下或向右移动, 问该数字矩阵从最左上角到最右下角的最小路径和是多少? (路径和就是将某路径中的所有权值全部加起来的总和)。

2. 算法实例:

1	3	5	7
8	6	4	2
5	0	1	3
4	8	7	2

选择 F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第二次上机课\3-2.exe

```
原数组:
1  3  5  7
8  6  4  2
5  0  1  3
4  8  7  2
最小权重数组为:
1  4  9 16
9 10 13 15
14 10 11 14
18 18 18 16
最小权重路径为:
(3, 3) <- (3, 2) <- (2, 2) <- (1, 2) <- (1, 1) <- (1, 0) <- (0, 0)
最小权重为: 16
最大权重数组为:
1  4  9 16
9 15 19 21
14 15 20 24
18 26 33 35
最大权重路径为:
(3, 3) <- (2, 3) <- (1, 3) <- (0, 3) <- (0, 2) <- (0, 1) <- (0, 0)
最大权重为: 35
请按任意键继续. . .
```

输入权重数组为 $[[1, 3, 5, 7], [8, 6, 4, 2], [5, 0, 1, 3], [4, 8, 7, 2]]$

输出: 16

3.程序代码:

/*动态规划算法之-----走方格 */

```
#include<stdio.h>
```

```
#include<string.h>
```

```
#include<stdlib.h>
```

```
int getMin(int map[4][4],int n,int m);
```

```
int Min(int a,int b); //系统有 min,max 函数
```

```
int getMax(int map[4][4], int n, int m);
```

```
int Max(int a, int b);
```

```
int main()
```

```
{
```

```
    int i=0,j=0;
```

```
    int array[4][4]={
```

```
        {2, 3, 0, 8},
```

```
        {4, 2, 7, 3},
```

```
        {5, 3, 0, 5},
```

```
        {0, 1, 2, 4}
```

```
    };
```

```
    printf("原数组: \n" );
```

```
    for (i = 0; i<4; i++){
```

```
        for (j = 0; j<4; j++){
```

```
            printf("%4d", array[i][j]);
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    printf("最小权重为: %d\n",getMin(array, 4, 4));
```

```
    printf("最大权重为: %d\n", getMax(array, 4, 4));
```

```
    system("pause");
```

```
    return 0;
```

```
}
```

```
int getMin(int map[4][4], int n, int m) {
```

```
    int dp[4][4] = {0};
```

```
    int i, j;
```

```
    for (i = 0; i<n; i++){ //第一行初始化, 一直向右走.
```

```
        for (j = 0; j <= i; j++){ //i=0,j=0 dp[0][0] i=0,j=0,1 dp[1][0]
```

```
            dp[i][0] += map[j][0];
```

```
        }
```

```
    }
```

```
    for (i = 1; i<m; i++){ //第一列初始化, 一直向下走; 如 i=0, 则 dp[0][0]会计算两次。
```

```

        for (j = 0; j <= i; j++){
            dp[0][i] += map[0][j];
        }
    }
    for (i = 1; i<n; i++){
        for (j = 1; j<m; j++){
            dp[i][j] = Min(dp[i][j - 1] + map[i][j], dp[i - 1][j] + map[i][j]);
        }
    }
    printf("最小权重数组为: \n");
    for (i = 0; i<n; i++){
        for (j = 0; j<m; j++){
            printf("%4d", dp[i][j]);
        }
        printf("\n");
    }
    printf("最小权重路径为: \n");
    i = 3; j = 3;
    while (i != 0 || j != 0){ //反向查询路径，可用栈存储，出栈即为正向路径
        printf("(%d, %d)", j, i);
        printf("<-");
        if (dp[i][j]-map[i][j]== dp[i - 1][j] ){
            i--;
        }
        else if (dp[i][j]-map[i][j]== dp[i][j - 1]){
            j--;
        }
    }
    printf("(%d, %d)\n", j, i);

    return dp[n - 1][m - 1];
}

int Min(int a, int b){
    if (a>b){
        return b;
    }
    else{
        return a;
    }
}

int getMax(int map[4][4], int n, int m) {
    int dp[4][4] = { 0 };
    int i, j;
    for (i = 0; i<n; i++){ //第一行初始化，一直向右走
        for (j = 0; j <= i; j++){
            dp[i][0] += map[j][0];

```



```

    }
}
for (i = 1; i < m; i++) { //第一列初始化，一直向下走；如 i=0，则 dp[0][0]会计算两次。
    for (j = 0; j <= i; j++) {
        dp[0][i] += map[0][j];
    }
}
for (i = 1; i < n; i++) {
    for (j = 1; j < m; j++) {
        dp[i][j] = Max(dp[i][j - 1] + map[i][j], dp[i - 1][j] + map[i][j]);
    }
}
printf("最大权重数组为: \n");
for (i = 0; i < n; i++) {
    for (j = 0; j < m; j++) {
        printf("%4d", dp[i][j]);
    }
    printf("\n");
}
printf("最大权重路径为: \n");
i = 3; j = 3;
while (i != 0 || j != 0) { //反向查询路径，可用栈存储，出栈即为正向路径
    printf("(%d, %d)", j, i);
    printf("<-");
    if (dp[i][j] - map[i][j] == dp[i - 1][j]) {
        i--;
    } else if (dp[i][j] - map[i][j] == dp[i][j - 1]) {
        j--;
    }
}
printf("(%d, %d)\n", j, i);
return dp[n - 1][m - 1];
}
int Max(int a, int b) {
    if (a > b) {
        return a;
    }
    else {
        return b;
    }
}
}

```

4.程序验证:

```
int array[4][4] = {
    {2, 3, 0, 8},
    {4, 2, 7, 3},
    {5, 3, 0, 5},
    {0, 1, 2, 4}
};
```

输入的原数组为{2, 3, 0, 8},{4, 2, 7, 3},{5, 3, 0, 5},{0,1, 2, 4}输出结果为最小权重为：16，结果如图所示：

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第二次上机课\3-2.exe

```
原数组:
2 3 0 8
4 2 7 3
5 3 0 5
0 1 2 4
最小权重数组为:
2 5 5 13
6 7 12 15
11 10 10 15
11 11 12 16
最小权重路径为:
(3, 3)<-(2, 3)<-(2, 2)<-(1, 2)<-(1, 1)<-(1, 0)<-(0, 0)
最小权重为: 16
最大权重数组为:
2 5 5 13
6 8 15 18
11 14 15 23
11 15 17 27
最大权重路径为:
(3, 3)<-(3, 2)<-(3, 1)<-(2, 1)<-(1, 1)<-(0, 1)<-(0, 0)
最大权重为: 27
请按任意键继续. . .
```

5.算法分析:

1. 确定 dp 数组以及下标的含义

dp[i][j] : 从 (0, 0) 出发, 到(i, j), dp[i][j]存储路径和 (最小值或者最大值)。

2. 确定递推公式 状态转移方程 $dp[i][j] = \text{Min}(dp[i][j - 1] + \text{map}[i][j], dp[i - 1][j] + \text{map}[i][j]);$

3. dp 数组初始化

4. 确定遍历顺序

- 时间复杂度: $O(m \times n)$
- 空间复杂度: $O(n)$

实验 2-5 线性时间选择

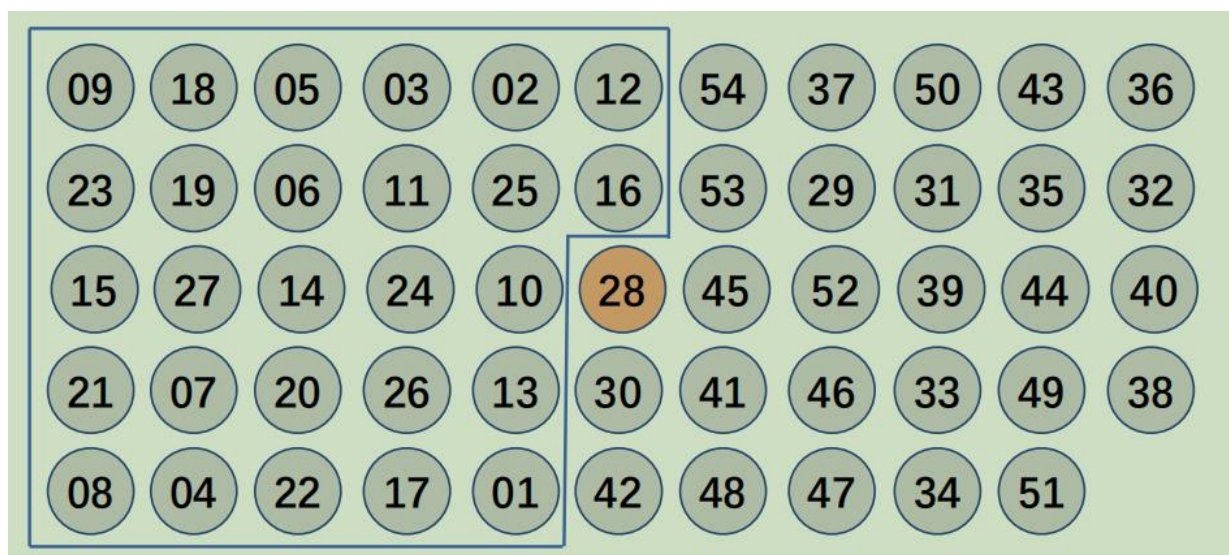
1.问题描述:

给定线性序集中 n 个元素和一个整数 k , $1 \leq k \leq n$, 要求找出这 n 个元素中第 k 小的元素。

2.算法实例:



$n=54, k=15$ 递归初始条件为 $n>5$. 以 5 个为一组, 取出每组的中位数, 构造中位数数组, 在每组排序的时间复杂度为 $O(1)$, 取出中位数数组 $[28,32]$ 的中位数 28 作为 pivot 对中位数数组 $[15,23,\dots,43]$ 进行 partition 将 MoM 作为 pivot 进行 $O(n)$ partition, 一轮 partition 后 pivot 排好序, 以此类推, 直至完成第 k 次排序, 取到第 k 小元素。



3.程序代码:

```
/**
 * 线性时间选择
 */
```

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>

#define MAX 100

void ssort(int num[], int p, int r);
void swap(int *x, int *y);
int Select(int num[], int p, int r, int k);

int main()
{
    int i = 19, k;
    int num[MAX] = { 6, 5, 9, 4, 3, 10, 2, 1, 7, 11, 8, 12, 18, 20, 13, 19, 15, 14, 17, 16};
    printf("请输入要选择第几小的元素: ");
    scanf("%d", &k); // 选择第几小的元素
    if (k > i)
        printf("error!\n");
    else
        printf("%d\n", Select(num, 0, i, k));
    system("pause");
    return 0;
}

/* 选择排序 */
void ssort(int num[], int p, int r)
{
    for (int i = p; i < r - 1; i++)
    {
        int iMin = i;
        for (int j = i + 1; j < r; j++) {
            if (num[j] < num[iMin]) {
                iMin = j;
            }
        }
        if (iMin != i) {
            swap(&num[i], &num[iMin]);
        }
    }
}

void swap(int *x, int *y)
{
    int temp;

```

```

    temp = *x;
    *x = *y;
    *y = temp;
}
// 分解函数
int Partition(int num[], int p, int r, int mid)
{
    int i = p, j = r;
    while (i <= r && j >= p)
    {
        while (num[i] < mid){ i++; }
        while (num[j] > mid){ j--; }
        if (i >= j)
            break;
        else
        {
            swap(&num[i], &num[j]);
            i++, j--;
        }
    }
    return j;
}
// 选择函数
int Select(int num[], int p, int r, int k)
{
    //用某个简单排序算法对数组 a[p:r]排序,直接返回第 k 小元素
    if (r - p < 75)
    {
        slsort(num, p, r); //排序
        return num[p + k - 1]; //返回
    }
    // 选出 n/5 组中每个组的中位数,将 a[p+5*i]至 a[p+5*i+4]的第 3 小元素与 a[p + i]交换位置
    for (int i = 0; i <= (r - p - 4) / 5; i++)
    {
        slsort(num, p + 5 * i, p + 5 * i + 4); //排序
        swap(&num[p + 5 * i + 2], &num[p + i]); //交换
    }
    // 选出各种中位数的中位数 mid, r-p-4
    int mid = Select(num, p, p + (r - p - 4) / 5, ((r - p - 4) / 5 + 1) / 2);
    // 以 mid 为基准进行分解
    int mid_id = Partition(num, p, r, mid);
    int mid_rank = mid_id - p + 1;
    // 递归条件判断
    if (k == mid_rank)
    {
        return num[mid_id];
    }
}

```

```

else if (k < mid_rank)
{
    return Select(num, p, mid_id, k);
}
else
{
    return Select(num, mid_id + 1, r, k - mid_rank);
}
}

```

4.程序验证：

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第二次上机课\2-5.exe

请输入要选择第几小的元素：6

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第二次上机课\2-5.exe

请输入要选择第几小的元素：8

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第二次上机课\2-5.exe

请输入要选择第几小的元素：3

分别验证排序为第 6、8、3 小的元素，输入的无序数组为 { 6, 5, 9, 4, 3, 10, 2, 1, 7, 11, 8, 12, 18, 20, 13, 19, 15, 14, 17, 16 } 输出结果为 6, 8, 3 结果如图所示：

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第二次上机课\2-5.exe

请输入要选择第几小的元素：6

6

请按任意键继续. . .

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第二次上机课\2-5.exe

请输入要选择第几小的元素：8

8

请按任意键继续. . .

F:\课程\第七学期\算法设计与分析\蔡艺洵的上机\第二次上机课\2-5.exe

请输入要选择第几小的元素：3

3

请按任意键继续. . .

5.算法分析：

算法将每一组的大小定为 5，并选取 75 作为是否作递归调用的分界点。这 2 点保证了 $T(n)$ 的递归式中 2 个自变量之和 $n/5 + 3n/4 = 19n/20 = \varepsilon n$ ， $0 < \varepsilon < 1$ 。这是使 $T(n) = O(n)$ 的关键之处。当 $n < 75$ 时， $T(n) = C1$ ， $n \geq 75$ 时， $T(n) = C2 * n + T(n/5) + T(3n/4)$ 。

$T(n) = O(n)$

分析与总结：

动态规划算法与分治法类似，将待求解问题分解成若干个子问题。分解得到的子问题往往不是互相独立的。子问题被重复计算了多次。通过保存已解决的子问题的答案，而在需要时再找出已求得的答案，就可以避免大量重复计算，从而得到多项式时间算法。

一般步骤为：

- 问题结构分析

给出问题表示，明确原始问题

- 递推关系建立

分析最优（子）结构，构造递推公式

- 自底向上计算

确定计算顺序，依次求解问题

- 最优方案追踪

记录决策过程，输出最优方案

本 次 上 机 成 绩	项目	操作	报告书写	出勤和课堂纪律	其他
	得分				
	成绩合计：				

评语：

教师签字：

日期：